Revisión del generador Quarkus tras las plantillas Jinja

Proyecto agentIA_v2 | Commit revisado: 9eb82ce (y 3d8a43d) | Octubre 2026

Veredicto

Las plantillas deterministas son un avance real y cubren la mejora principal recomendada. Sin embargo, la afirmación de que el código generado es "altamente confiable y estándar de producción" es exagerada: no hay evidencia de que lo generado compile y arranque, y existen problemas de configuración y diseño que impiden llamarlo listo para producción. Los 194 tests de Python pasan, pero validan el generador, no el servicio Quarkus resultante.

Lo que sí se cumple

- **CRUD determinista:** Entity, Repository y Resource salen de plantillas Jinja con PanacheRepository, @Transactional en escritura y códigos HTTP correctos (201, 200, 404, 204).
- **PUT corregido:** carga la entidad existente y actualiza campos en lugar de persist(updated).
- **Namespace Jakarta:** no se usan javax.* y los tipos monetarios vienen como BigDecimal desde el blueprint.

Lo que no es cierto o no está demostrado

- **"BOM oficial de Red Hat":** el BOM es io.quarkus.platform:quarkus-bom, la plataforma comunitaria de Quarkus.
- **Que compile y corra:** no existe ninguna prueba de mvn package ni de arranque del servicio generado.
- **"Estándar de producción":** ver los problemas de la tabla siguiente.

Problemas detectados

#	Problema	Dónde	Severidad
1	application.properties usa quarkus.hibernate-orm.database.generation=drop-and-create: borra la base de datos en cada arranque. Además define db-kind sin URL, usuario ni password.	app/orchestrator/generation.py: 207	Alta
2	El pom.xml se arma a mano en Python, con versión 3.15.1 fija y hibernate-validator siempre incluido; ignora el scaffolder oficial y el extension_map del perfil. Hay dos fuentes de verdad.	app/orchestrator/generation.py: 62-190	Alta
3	@NotNull sin @Valid en el recurso: una entrada incompleta produce 500 en el flush en vez de 400.	templates/quarkus/Resource.java.jinja	Media
4	Clave primaria asumida como Long. Una PK UUID o String no compila. Se ignora primaryKey del blueprint y solo se detecta un campo llamado id (otro nombre agrega un id extra).	template_engine.py / plantillas	Media
5	render_template_if_available captura toda excepción y devuelve None: un bug de plantilla cae al LLM sin dejar rastro.	agents/generator/template_engine.py	Media

#	Problema	Dónde	Severidad
6	ResourceTest solo prueba GET lista y 404; no cubre POST, PUT ni DELETE.	templates/quarkus/ResourceTest.java.jinja	Media
7	POST con id enviado por el cliente: persist con IDENTITY falla (detached entity). Poner el id a null.	templates/quarkus/Resource.java.jinja	Baja
8	La entidad se expone directamente (sin DTO) y listAll() no pagina.	templates/quarkus/Resource.java.jinja	Baja
9	El Dockerfile no copia agents/, profiles/ ni templates/: la imagen no funcionaría con el código actual.	Dockerfile	Media
10	No existe un smoke test end-to-end (mvn package, levantar, llamar a un endpoint).	tests/	Alta

Plan recomendado

Fase	Contenido	Esfuerzo
Rápida	Puntos 5, 7 y 6: loguear fallos de plantilla, anular el id en POST y ampliar el test con POST, PUT y DELETE.	Bajo
Importante	Puntos 1 y 2: derivar el pom.xml del extension_map del perfil y separar application.properties para %dev/%test y producción (sin drop-and-create en producción).	Medio
Verificación	Punto 10: smoke test con demo_quarkus.json que ejecute mvn verify y llame a un endpoint. Es la única evidencia real de confiabilidad.	Medio
Después	Puntos 3, 4, 8 y 9: @Valid, PK configurable, DTOs/paginación y Dockerfile.	Medio

Nota sobre el alcance

Esta revisión se basa en la lectura del código y en la ejecución de la suite de tests de Python (194 pasados). No se compiló ni se levantó ningún servicio generado, ya que Java y Maven no están verificados en este equipo.